.net Core Course Content: -

1. Introduction
2. .net F/W vs .net Core
3. Folder structure
4. MVC Architecture
5. Model, View, Controller
6. Controller and Action Methods
7. Views and Types
8. HTML Helpers vs Tag Helpers
    a. Standard HTML Helpers
    b. Strongly Typed HTML Helpers
9. Asynchronous Programming
    a. async/await
10. Adv C#.net
    a. Delegates
    b. ExtensionMethods
    c. LambdaExpressions
    d. Object Initializer
    e. Collection Initializer
    f. Generics
    g. Null Coalescing Assignment
    h. Null Coalescing Operator
    i. Nullable DataTypes
    j. Linq Queries
11. Dependency Injection in C#
    a. Constructor injection
    b. Scoped, Transient, Singleton lifetimes
    c. DI best practices in .NET Core
12. MiddleWares in ASP.net Core
    a. HttpHandlers vs HttpModules
    b. MiddleWare Execution Order
    c. Use,Run,Map methods

d. Custom Middleware

13. State Management Techniques in .net Core
    a. View Data
    b. ViewBag
    c. TempData
14. Object Life Cycle in .net Cor(Transient,Scoped,Singleton)
15. DataBinding in .net Core
16. Crud Operations with ADO.net
17. Crud Operations with ADO.net – Stored Procedures
18. Entity Framework
    a. DataBase First Approach
    b. Code First Approach
19. EF Core
    a. Entity F/W vs EFCore
    b. EF Core Approachs
    c. EF Core Migrations
    d. Crud Operations with EF Core
    e. Relationships between Entities in EF Core
20. Data Annotations in ASP.net Core
21. Authentication and Authorization
    a. O-Auth & Social Authentication
    b. JWT
    c. Access Token
22. Filters in ASP.net Core
23. Service Oriented Architecture
24. WCF vs WEB API
25. **WebAPI**
    a. Architecture
    b. API controller vs MVC controller
    c. Routing & attribute routing
    d. Model Binding
    e. Model Validation
    f. Returning Action Results

g. Versioning APi's
h. WebAPi with EFCore
i. API returning JSON

26. Http Methods

a. HttpGet
b. HttpPost
c. HttpPut
d. HttpDelete

27. CORS (Cross-Origin Resource Sharing)

a. Need for CORS
b. Enabling CORS
c. Policy-based CORS
d. CORS with Angular

28. How to Test WebAPI

a. PostMan
b. Swagger

29. Caching

a. Response caching
b. In-memory caching
c. Distributed caching
d. Redis caching
e. Cache invalidation

30. Calling External APIs

31. Deployment

a. Publish Web API
b. Deploy to IIS
c. Deploy to Azure App Service
d. CI/CD with GitHub Actions or Azure DevOps

📘 MICROSERVICES SYLLABUS (Beginner → Advanced):-

1. **Introduction to Microservices**
    a. What are Microservices?
    b. Monolithic vs Microservices Architecture
    c. Characteristics of Microservices
    d. Advantages & Disadvantages
    e. When to use Microservices
    f. Real-world industry examples
2. **Core Microservices Concepts**
    a. SOLID Principles
    b. Synchronous vs Asynchronous communication
    c. Stateless services
    d. Service registry & discovery
    e. API Gateway pattern
    f. Fault tolerance
3. **Setting Up the Development Environment**
4. **RESTful APIs in Microservices**
    a. REST API fundamentals
    b. HTTP Verbs
    c. Request/Response model
    d. Designing standard APIs
    e. Data transfer objects (DTOs)
    f. Versioning APIs
5. **NET Core Microservices Development**
    a. Creating API's
    b. Models, DTOs, Automapper
    c. Dependency Injection
    d. Repository Pattern
    e. EF Core for data access
    f. Async programming in services
    g. Building multiple microservices (Product, Order, Payment, etc.)

6. **Communication Between Microservices**
    a. Synchronous
    b. Asynchronous
7. **API Gateway**
    a. What is an API Gateway?
    b. Why we need it
    c. Routing, aggregation, filtering
    d. Rate limiting
    e. Authentication at gateway
    f. Ocelot API Gateway (.NET)
8. **Configuration Management**
    a. appsettings.json
    b. yaml files
    c. Azure app configuration
9. **Service Discovery**
    a. Eureka
    b. Consul
10. **Microservices Security**
    a. **Authentication**
    b. **Authorization**
    c. **OAuth**
11. **CQRS (Command Query Responsibility Separation)**
12. **Containerization with Docker**
13. **CI/CD for Microservices**
    a. Using GitHub Actions
    b. Using Azure DevOps
14. **Deploying Microservices**
    a. Deploy to IIS (Windows-based hosting)
    b. Deploy to Azure App Service
15. **Testing Microservices(PostManSwagger)**

16. **Design Patterns :-**
    a. Saga Pattern
    b. Clean Architecture
    c. Singleton Pattern
    d. Factory Pattern
    e. Repository Pattern
    f. CQRS (Command Query Responsibility Segregation)
    g. Observer Pattern
17. **Hands-on Microservices Capstone Project**
    a. A complete project with:
    b. Product Service
    c. Order Service
    d. Payment Service
    e. Notification Service
    f. API Gateway
    g. RabbitMQ/Kafka for async messaging
    h. Dockerized services
    i. deployment
    j. CI/CD pipeline

Azure for .net Core:-

1. Introduction to Azure

- What is Microsoft Azure?
- Azure Global Infrastructure (Regions, Zones)
- Azure Resource Manager (ARM)
- Azure Portal, CLI, PowerShell
- SAAS,PAAS,IAAS

2. Creating Azure Account

3. Azure Essential Modules

a. Appservices
b. Repos
c. Storage Accounts
d. Azure SQl DataBase
e. Azure Devops-CI/CD
f. Azure Service Bus
g. Azure functions

4. Azure AppServices:-

a. Host .NET Core APIs, MVC
b. App Service Plan (Free, Basic, Standard)
c. Deployment methods
d. Application settings

5. Azure Repos (Git Repository)

a. Creating repositories
b. Push/Commit workflow
c. Branching strategies
d. Pull Requests
e. Integrating with Azure Pipelines

6. Azure Storage Accounts

1. **Angular Syllabus: -**
    a. Introduction to Angular
    b. What is Angular?
    c. Angular vs React vs Vue
    d. Angular Architecture
    e. SPA (Single Page Application)
    f. Angular CLI Overview
2. **TypeScript Fundamentals**
    a. Introduction to TypeScript
    b. Data Types
    c. Interfaces & Classes
    d. Modules & Namespaces
    e. Access Modifiers
    f. Arrow Functions
    g. Generics
    h. Decorators
    i. Enums
3. **Angular Project Setup**
    a. Installing Angular CLI
    b. Creating a new Angular project
    c. Project folder structure
    d. Running Angular project
    e. Understanding angular.json
4. **Components**
    a. Creating Components
    b. Component Decorator
    c. Templates & Styles
    d. Component Communication
    e. @Input
    f. @Output + EventEmitter
    g. ViewChild & ContentChild
    h. Lifecycle Hooks
    i. ngOnInit

5. **Data Binding**
    a. String Interpolation
    b. Property Binding
    c. Event Binding
    d. Two-way Binding (ngModel)
    e. Template Reference Variables
6. **Directives**
    Structural Directives
    a. *ngIf
    b. *ngFor
    c. *ngSwitch
    Attribute Directives
    d. ngClass
    e. ngStyle
    f. Custom Directives
7. **Pipes**

- Built-in Pipes
    - DatePipe
    - CurrencyPipe
    - JSON Pipe
    - Slice Pipe
- Custom Pipes

8. **Services & Dependency Injection**
    a. What are Services?
    b. Creating a Service
    c. Dependency Injection (DI)
    d. Hierarchical Injectors
    e. Service Providers
    f. Singleton Services
    g. Using Services in Components
    h. Observable-based services

9. **Routing & Navigation**
    a. Introduction to Angular Routing
    b. Configuring Routes
    c. RouterModule & ForRoot
    d. RouterLink & RouterLinkActive
    e. Navigating Programmatically
10. **Forms in Angular**

    Template-Driven Forms

    a.FormsModule
    b. ngModel
    c. Built-in Validators
    d. Custom Validators

    Reactive Forms

    a. FormControl
    b. FormGroup
    c. FormBuilder
    d. Reactive Validation
11. **HTTP Client & API Integration**
    a. HttpClientModule
    b. GET, POST, PUT, DELETE requests
    c. Handling Errors
    d. Observables & RxJS
    e. Http Interceptors

    JWT Token Interceptor

    a. Handling API Headers
    b. Environment Variables
    c. Connecting to real backend (.NET APIs)
12. **RxJS in Angular**

    a. What is RxJS?

    b. Observable & Observer

    c. Subjects & BehaviorSubject

d. Operators

- map
- filter
- switchMap
- debounceTime
- take / takeUntil

e. AsyncPipe

f. Converting Observables to Promises

g. Memory leaks & unsubscribe patterns

13. Angular State Management

a. Local Component State

b. Service-based State Sharing

c. BehaviorSubject for State Handling

d. Introduction to NgRx

e. Actions

f. Reducers

g. Effects

h. Store

14. Authentication & Authorization

a. JWT Authentication

b. Login & Register UI

c. Token Interceptor

d. Storing tokens (LocalStorage/SessionStorage)

e. Protecting routes using guards

g. Logout mechanism

TRAIN YOUR BRAIN
MygoMINDS
DFSD
.NET
Core
Azure